Day 2

Images, Compose & webMethods Stacks

Build your own images, then run MSR + Universal Messaging + PostgreSQL with Compose

Block 3 · Building images & Compose

Block 4 · webMethods multi-container stacks

Day 2: Agenda

Block 3 · 09:00–13:00

- Anatomy of a Containerfile, build context
- Layer order and caching
- CMD vs. ENTRYPOINT, multi-stage
- Registries: tag, push, pull
- The MSR Containerfile, base vs. custom
- Compose: services, networks, volumes
- 🏆 From commands to one compose.yaml

Block 4 · 13:00–17:00

- Compose for MSR
- Universal Messaging in Compose
- MSR ↔ UM via application.properties
- MSR + PostgreSQL: JDBC pools and drivers
- Healthcheck-gated startup, readiness vs. liveness
- 🏆 Capstone 4: MSR + UM + PostgreSQL
- Day 2 recap

Hands-on: *Exercises 3.1–3.7 and 4.1–4.2 · Capstone 3 (40 min) · Capstone 4 (115 min)*

Classification - Internal



Block 3

Building Images & Docker Compose

Containerfiles · Layers · Multi-stage · Registries · Custom MSR image · Compose

🔧 Block 3 Exercises: Build and Compose

- 3.1 Build your first image (and where HEALTHCHECK goes)
- 3.2 Layer order decides build time; .containerignore
- 3.3 CMD vs. ENTRYPOINT
- 3.4 Multi-stage build: compiler in the kitchen, not the dining room
- 3.5 Ship it: tag, push, pull through a registry; save / load
- 3.6 A custom MSR image, and build secrets that do not leak
- 3.7 Your first compose file
- 🏆 Capstone 3: from podman run commands to one compose.yaml

Classification - Internal

Anatomy of a Containerfile

Instruction	Purpose	Example
FROM	Base image	FROM eclipse-temurin:17-jre-alpine
RUN	Execute during build	RUN apk add --no-cache curl
COPY	Copy files into image	COPY target/app.jar /opt/app.jar
ENV	Set env variables	ENV JAVA_OPTS="-Xmx512m"
EXPOSE	Document port (not publish)	EXPOSE 5555
WORKDIR	Set working directory	WORKDIR /opt/softwareag
USER	Set runtime user	USER 1724
ENTRYPOINT	Main process (exec form)	ENTRYPOINT ["java","-jar","app.jar"]
HEALTHCHECK	Container health probe	HEALTHCHECK CMD curl -f localhost/health
CMD	Main command (exec form) – can be overridden	CMD ["python", "app.py"]

Classification - Internal

Understanding Docker Build Context

What is Build Context?

Set of files and directories sent to the container engine during image build.

How It Works

```
podman build -t myimage:1.0 .
```

- The '.' specifies the build context (current directory)
- All files in this directory are sent to the build daemon
- COPY and ADD instructions can only access files within the context

Context Scope Rules

✓ COPY ./app /opt/app
✓ COPY config/*.xml /etc/config/
✗ COPY ../outside/file.txt /app/
✗ COPY /absolute/path/file.txt /app/

Optimizing with .containerignore

Exclude files from the build context to reduce build time and image size:

```
# .containerignore example
node_modules/     # Dependencies (reinstall in image)
.git/             # Version control history
*.log             # Log files
.env              # Secrets (never include!)
```


Classification - Internal

Build Best Practices: Layer Ordering

`FROM base:tag`	*Rarely changes*
`RUN install-system-packages`	*Rarely changes*
`COPY dependency-manifest`	*Changes sometimes*
`RUN install-dependencies`	*Cached if above unchanged*
`COPY application-code`	*Changes every build*
`CMD / ENTRYPOINT`	*Rarely changes*

Classification - Internal

Image & Layer Gotchas and Best Practices

Gotchas & pitfalls

- ✗ Changing one early layer invalidates the cache for every layer after it
- ✗ The latest tag is not a version; it silently drifts („floating tag“)
- ✗ A secret copied into an early layer stays in the image history even if a later layer deletes it
- ✗ Container writes vanish on podman rm unless they're on a volume

Best practices

- ✓ Order instructions from least to most frequently changing
- ✓ Pin explicit tags (...:12.1.0.3) or digests (@sha256:...) for base images
- ✓ Never COPY credentials or licenses; inject them at runtime
- ✓ Logs, UM data and DB data that must survive belong on volumes

Classification - Internal

🔧 Hands-On: Exercises 3.1–3.2

3.1	Build my-web, read podman image history; HEALTHCHECK only survives --format docker
3.2	Containerfile.slow vs. .fast after a change (8 s vs. 1 s); .containerignore keeps .env out
📁 **Files**	block3/exercises/my-web, block3/exercises/layer-order

Participant Workbook, Block 3 · ~30 min · 🏁 you can state the layer-order rule in one sentence

Classification - Internal

CMD vs. ENTRYPOINT

CMD (Can Be Overridden)

Provides default arguments that can be replaced at runtime:

```
CMD ["python", "app.py"]

podman run myimage                # Runs: python app.py
podman run myimage python test.py # Runs: python test.py
                                          (CMD replaced)
```

ENTRYPOINT (Fixed Command - Cannot be Overridden)

Sets the main executable that always runs (arguments can be appended):

```
ENTRYPOINT ["python", "app.py"]

podman run myimage          # Runs: python app.py
podman run myimage --debug # Runs: python app.py
                                  #    --debug (args
                                  #    appended)
```

Combine Both

Use ENTRYPOINT for the executable, CMD for default arguments:

```
ENTRYPOINT ["python"]
CMD ["app.py"]

podman run myimage                    # Runs: python app.py
podman run myimage test.py            # Runs: python test.py (CMD replaced)
podman run myimage app.py --debug     # Runs: python app.py --debug
```

Multi-Stage Builds

Stage 1: Build (Kitchen)

```
FROM maven:3.9 AS builder
COPY pom.xml + src/
RUN mvn package

Includes: Maven, JDK, source, tests
Size: ~850 MB
```

Stage 2: Runtime (Dining Room)

```
FROM eclipse-temurin:17-jre-alpine
COPY --from=builder app.jar
USER app
ENTRYPOINT ["java","-jar",...]

Includes: Only JRE + compiled JAR
Size: ~180 MB
```



Classification - Internal

🔧 Hands-On: Exercises 3.3–3.5

3.3	ENTRYPOINT + CMD: run with no args, with args, with --entrypoint
3.4	Java Hello: single-stage (JDK, ~520 MB) vs. multi-stage (JRE, ~360 MB, no javac)
3.5	Local registry on :5001: tag, push, list tags, pull back; podman save / load
📁 Files	block3/exercises/entrypoint, block3/exercises/multistage; registry:2 on port 5001

Participant Workbook, Block 3 · ~35 min · 🏁 my-web:1.0 listed in the registry catalog

Classification - Internal



IBM webMethods Microservices Runtime

Build

MSR: Typical Containerfile

```
FROM ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
```

Base Image

```
RUN --mount=type=secret,id=wpm_token \
RUN /opt/webmethods/wpm/bin/wpm.sh install -ws https://packages.webmethods.io -wr licensed -j "$(cat /run/secrets/wpm_token)" WmJDBCAdapter -d /opt/webmethods/IntegrationServer
```

Build secret: an ARG would stay in the image history (Ex. 3.6)

webMethods Package Manager for installing additional packages

```
COPY --chown=1724:1724 libs/jdbc/ /opt/webmethods/IntegrationServer/packages/WmJDBCAdapter/code/jars/
```

Add dependent jar files / libs from host

```
EXPOSE 5555 5543 9999
```

Export ports

```
ENTRYPOINT ["/opt/webmethods/IntegrationServer/bin/startContainer.sh"]
```

Default Entrypoint

Classification - Internal

Base Images vs. Custom Images

Base image (from IBM)

- Product runtime only: no customer packages, no licence, no JDBC drivers
- Updated by IBM with fixes; you choose when to move the tag
- Used as FROM in your Containerfile
- Never patched by hand: extend it instead

Custom image (yours)

- FROM ibmwebmethods.azurecr.io/webmethods-microservicesruntime:12.1.0.3
- Adds JDBC drivers + driver aliases, your packages, a HEALTHCHECK
- Built and versioned by GitLab CI (Day 3), pushed to the Swisscom registry
- The artefact that is promoted Dev → Test → Prod unchanged

Rule of thumb: if IBM ships it, extend it in a layer. If you own it, bake it into your image. Configuration and secrets come from outside.

Classification - Internal

Container Image Gotchas & Best Practices

Gotchas & pitfalls

- ✗ A licence file COPY'd into a layer leaks to everyone who can pull the image
- ✗ An unpinned base tag makes a routine rebuild jump a fix level
- ✗ Poor layer order makes every CI build slow and discourages frequent rebuilds
- ✗ Hand-built images that skip IBM's base lose vendor support

Best practices

- ✓ Mount the licence at runtime and point SAG_IS_LICENSE_FILE at it
- ✓ Pin tags or digests; bump deliberately in a reviewed change
- ✓ Drivers early, packages last; mirror base images in the Swisscom registry
- ✓ Always build FROM the IBM-provided image

Classification - Internal

🔧 Hands-On: Exercise 3.6

3.6 A	Custom MSR image: COPY --chown=1724:1724 the SwisscomLab package; run it; health info answers
3.6 B	--build-arg token shows up in podman history; --secret + RUN --mount=type=secret does not
📁 **Files**	block3/exercises/msr-custom (build context = repository root)

Participant Workbook, Block 3 · ~25 min · 🏁 the ARG token is readable, the build secret is not

Classification - Internal



Docker Compose / Podman Compose

Declaratively define multi-container applications

Podman Compose: Multi-Container Development

What is Podman Compose?

A tool for defining and running multi-container applications using YAML configuration

- Compatible with Docker Compose file format (compose.yaml)
- Manages container lifecycle: build, start, stop, networking, volumes
- Single command to orchestrate entire application stack

✓ Good For (Development & Testing)

- Local development environments (laptop/workstation)
- Quick prototyping and testing multi-container apps
- CI/CD pipeline integration tests
- Simple single-host deployments (dev/staging servers)

✗ Not Good For (Production at Scale)

- Multi-host / distributed deployments
- High availability and auto-scaling requirements
- Advanced orchestration (rolling updates, health checks, secrets)
- Enterprise production workloads

Classification - Internal

Compose: The Multi-Container Menu

compose.yaml Structure

```
services:
  web:
    image: nginx:alpine
    ports: ["8080:80"]
    depends_on: [api]
  api:
    build: ./api
    environment:
      DB_HOST: db
  db:
    image: postgres:16
    volumes: [pgdata:/var/lib/...]
volumes:
  pgdata:
```

Key Commands

`podman-compose up -d`
Start all services (background)

`podman-compose down -v`
Stop & remove everything

`podman-compose logs -f api`
Follow specific service logs

`podman-compose up -d --build`
Rebuild images & restart

Classification - Internal

YAML Syntax & Structure: Sharp Edges

- Indentation is the only structure YAML has; mixing tabs and spaces breaks parsing with cryptic errors
- yes/no/on/off and numbers are auto-typed; quote values like "512" when a string is expected
- Anchors (&msr) and merge keys (<<: *msr) reuse a block, e.g. two identical MSR nodes in Capstone 5
- $$ escapes a literal $ (needed in healthchecks: pg_isready -U $${POSTGRES_USER})
- ${VAR:?message} fails fast when a required variable is missing; ${VAR:-default} supplies a default
- podman-compose config renders the fully resolved file: run it before every up when debugging

Best practice: Most "Compose is broken" issues are YAML or variable issues that podman-compose config exposes immediately.

Classification - Internal

Service Definition: Key Fields

Identity & build

- image: pull this exact tag
- build: context + dockerfile (our custom MSR image)
- image + build together: name the built image
- hostname: / container_name: (the latter blocks scaling)
- restart: no / on-failure:5 / unless-stopped

Runtime shape

- ports: host:container publishing
- environment: / env_file: configuration
- volumes: named volumes or bind mounts
- depends_on: with condition: service_healthy
- healthcheck: readiness probe
- networks:, secrets:, deploy.resources.limits:

Classification - Internal

Networks & Volumes in Compose

- Each project gets a default network; services resolve each other by service name (um, postgres, msr)
- Define explicit networks to segment traffic: in Capstone 5 only API Gateway reaches its OpenSearch store
- Named volumes under volumes: survive compose down; compose down -v deletes them
- Bind mounts (./config/application.properties) are perfect for configurations that live in Git
- Relative paths resolve against the project directory (the folder of the first -f file)
- external: true lets several projects share a network or volume

Classification - Internal

🔧 Hands-On: Exercise 3.7

3.7	The Exercise 1.6 bind-mounted site as compose.yaml: config, up, ps, logs, exec, down
📁 Files	block3/exercises/first-compose/compose.yaml (you write it)

Participant Workbook, Block 3 · ~15 min · 🏁 first_web_1 and network first_default appear and disappear

Classification - Internal

Compose Gotchas & Best Practices

Gotchas & pitfalls

- ✗ depends_on alone only waits for the container to START, not for MSR to be READY
- ✗ Relative bind-mount paths resolve against the project dir, not your shell's directory
- ✗ Variable precedence (shell > .env > defaults) surprises people
- ✗ podman-compose down -v deletes named volumes, including the only copy of dev data

Best practices

- ✓ depends_on with condition: service_healthy + a real readiness healthcheck
- ✓ Keep compose files, .env.example and mounted config together in Git
- ✓ podman-compose config before you debug anything else
- ✓ Treat named volumes as precious; back up before down -v

Classification - Internal

🏆 Capstone 3: From Commands to One compose.yaml

Objectives

40 min

- db: postgres:16, named volume, pg_isready healthcheck, Block 1 limits
- api: built from ./api, tagged localhost/workshop/shop-api:1.0
- api starts only when db is healthy (depends_on + condition)
- Passwords from .env, never in compose.yaml
- Prove it: visits survive down/up; down -v resets them
- Compare with your Block 1 script

Prerequisites

- Capstone 1 script at hand
- block3/capstone/
- Reference: block3/solution/compose.yaml

Deliverable
compose.yaml with the 6 TODOs done; the visits counts after down/up and after down -v, explained.

Classification - Internal

💬 Q&A and Discussion: Capstone 3

1. *Your Block 1 script vs. one compose.yaml: what got easier, and what can Compose not do that your script did?*

2. *Visits after down vs. down -v: which Swisscom data must never live where one flag can delete it?*

3. *Where does today's start order between IS, database and messaging rely on manual steps or luck?*

Classification - Internal